Proxmox VE 入門

aoki-taquan

2026 年 04 月 11 日

目次

1 はじめに 8
1.1 Proxmox VE とは 8
1.2 なぜ Proxmox VE を選ぶのか 8
1.2.1 他の仮想化プラットフォームとの比較 8
1.3 このガイドについて 9
1.4 対象読者 9
1.5 前提知識 9
2 インストール 10
2.1 システム要件 10
2.1.1 最小要件 10
2.1.2 推奨要件 10
2.2 ISO イメージのダウンロード 10
2.3 インストールメディアの作成 10
2.3.1 USB メモリへの書き込み 10
2.4 インストール手順 11
2.4.1 BIOS/UEFI の設定 11
2.4.2 インストーラーの起動 11
2.4.3 ディスクの設定 11
2.4.4 ネットワークの設定 11
2.4.5 パスワードとメールの設定 11
2.4.6 インストールの完了 11
2.5 初期設定 12
2.5.1 Web UI へのアクセス 12
2.5.2 サブスクリプションの無効化（任意） 12
2.5.3 タイムゾーンと NTP の確認 12
3 基本操作 13
3.1 Web UI の構成 13
3.2 ノードの管理 13
3.2.1 ノード情報の確認 13
3.2.2 シェルアクセス 13
3.3 コマンドラインツール 13
3.3.1 qm（仮想マシン管理） 13
3.3.2 pct（コンテナ管理） 14
3.3.3 pvesm（ストレージ管理） 14
3.3.4 pvesh（API シェル） 14
3.4 ISO イメージのアップロード 15
3.4.1 Web UI から 15
3.4.2 コマンドラインから 15
3.5 テンプレートのダウンロード 15

3.5.1 Web UI から . . . 15
3.5.2 コマンドラインから . . . 15
4 仮想マシン . . . 16
4.1 仮想マシンの作成 . . . 16
4.1.1 Web UI から作成 . . . 16
4.1.2 コマンドラインから作成 . . . 17
4.2 VirtIO ドライバー . . . 17
4.2.1 VirtIO とは . . . 17
4.2.2 Linux ゲスト . . . 17
4.2.3 Windows ゲスト . . . 17
4.3 QEMU Guest Agent . . . 18
4.3.1 有効化と導入 . . . 18
4.4 リソース配分のガイドライン . . . 18
4.4.1 CPU . . . 18
4.4.2 メモリ . . . 18
4.4.3 用途別の目安 . . . 19
4.5 スナップショット . . . 19
4.5.1 注意点 . . . 19
4.6 クローン . . . 19
4.6.1 フルクローン . . . 19
4.6.2 リンクドクローン . . . 20
4.7 テンプレート . . . 20
4.8 Windows VM のベストプラクティス . . . 20
4.8.1 Windows インストール手順 . . . 20
5 コンテナ（LXC） . . . 21
5.1 LXC コンテナとは . . . 21
5.2 コンテナの作成 . . . 21
5.2.1 Web UI から作成 . . . 21
5.2.2 コマンドラインから作成 . . . 22
5.3 コンテナの管理 . . . 22
5.3.1 基本操作 . . . 22
5.3.2 リソースの変更 . . . 22
5.4 特権コンテナと非特権コンテナ . . . 23
5.4.1 非特権コンテナ（推奨） . . . 23
5.4.2 特権コンテナ . . . 23
5.5 マウントポイント . . . 23
5.6 コンテナのバックアップと復元 . . . 23
6 ストレージ . . . 24
6.1 ストレージの概念 . . . 24
6.1.1 コンテンツタイプ . . . 24
6.2 ストレージの種類 . . . 24
6.2.1 ローカルストレージ . . . 24

6.2.2 共有ストレージ 25
6.3 ストレージの管理 26
6.4 推奨構成例 26
6.4.1 単一ノード構成 26
6.4.2 クラスタ構成 26
7 ネットワーク 27
7.1 ネットワークの基本構成 27
7.2 ブリッジの設定 27
7.2.1 Web UI から 27
7.2.2 設定ファイル 27
7.3 VLAN 28
7.3.1 VLAN Aware ブリッジ 28
7.4 ボンディング 28
7.5 ファイアウォール 29
7.5.1 ファイアウォールの有効化 29
7.5.2 ルールの追加 29
7.5.3 セキュリティグループ 29
7.6 SDN（Software Defined Networking） 30
7.6.1 主な概念 30
7.6.2 SDN の基本的な設定手順 30
8 バックアップとリストア 31
8.1 バックアップの種類 31
8.2 バックアップの実行 31
8.2.1 Web UI から 31
8.2.2 コマンドラインから 31
8.2.3 圧縮方式 31
8.3 スケジュールバックアップ 31
8.3.1 Web UI から設定 31
8.3.2 設定ファイル 32
8.4 リストア（復元） 32
8.4.1 Web UI から 32
8.4.2 コマンドラインから 32
8.5 Proxmox Backup Server（PBS） 32
8.5.1 主な特徴 32
8.5.2 PBS との連携設定 33
8.6 バックアップの保持ポリシー 33
9 クラスタ 34
9.1 クラスタとは 34
9.1.1 クラスタの利点 34
9.1.2 クラスタの要件 34
9.2 クラスタの構築 34
9.2.1 最初のノードでクラスタを作成 34

9.2.2 他のノードの参加 34
9.2.3 クラスタの確認 35
9.3 ライブマイグレーション 35
9.3.1 前提条件 35
9.3.2 マイグレーションの実行 35
9.4 高可用性（HA） 35
9.4.1 HA の設定 35
9.4.2 HA グループ 36
9.4.3 HA の状態とリソース管理 36
9.4.4 フェンシング 36
9.5 クォーラムの仕組み 36
9.6 2 ノードクラスタ 37
9.6.1 QDevice の導入（推奨） 37
9.6.2 QDevice なしの場合 37
9.7 クラスタのトラブルシューティング 37
9.7.1 クラスタの健全性確認 37
9.7.2 よくある問題 38
9.8 クラスタからのノード削除 38
10 Tips 40
10.1 Cloud-Init 40
10.1.1 基本設定 40
10.1.2 カスタム Cloud-Init 設定 40
10.1.3 Cloud-Init テンプレートのワークフロー 40
10.2 PCI パススルー 41
10.2.1 IOMMU の有効化 41
10.2.2 VFIO モジュールの設定 42
10.2.3 デバイスの割り当て 42
10.2.4 GPU パススルーの注意点 42
10.3 REST API の活用 42
10.4 Terraform との連携 43
10.5 タグとバルク操作 44
10.5.1 タグ 44
10.5.2 バルク操作 44
10.6 トラブルシューティング 44
10.6.1 よくある問題と対処法 44
10.7 便利なコマンド集 45
11 ユーザー管理と認証 46
11.1 認証レルム 46
11.1.1 レルムの追加（LDAP の例） 46
11.2 ユーザーの管理 46
11.2.1 ユーザーの作成 46
11.2.2 ユーザーの変更・削除 47

11.3 グループ 47
11.4 ロールと権限 47
11.4.1 組み込みロール 47
11.4.2 権限の付与 48
11.4.3 権限パスの構造 48
11.4.4 カスタムロールの作成 48
11.5 API トークン 48
11.6 リソースプール 49
11.7 ベストプラクティス 49
12 監視と通知 50
12.1 組み込みの監視機能 50
12.1.1 Web UI のダッシュボード 50
12.1.2 コマンドラインでの確認 50
12.2 通知システム 50
12.2.1 通知の仕組み 50
12.2.2 メール通知の設定 51
12.2.3 Matcher の設定 51
12.3 外部監視ツールとの連携 52
12.3.1 Prometheus + Grafana 52
12.3.2 SNMP 53
12.4 ヘルスチェックのベストプラクティス 53
13 アップデートとメンテナンス 54
13.1 リポジトリの管理 54
13.1.1 リポジトリの確認と設定 54
13.2 パッケージの更新 54
13.2.1 通常のセキュリティ更新 54
13.2.2 更新前の確認事項 54
13.2.3 カーネル更新後の再起動 55
13.3 メジャーバージョンアップグレード 55
13.3.1 アップグレード前の準備 55
13.3.2 アップグレード手順の概要 55
13.3.3 クラスタ環境でのアップグレード 55
13.4 ノードのメンテナンス 56
13.4.1 計画メンテナンス 56
13.4.2 ディスク交換（ZFS） 56
13.4.3 ディスク追加（ZFS プール拡張） 57
13.5 自動化 57
13.5.1 Unattended Upgrades 57
14 セキュリティ 58
14.1 SSL/TLS 証明書 58
14.1.1 デフォルトの自己署名証明書 58
14.1.2 Let’s Encrypt（ACME） 58

14.1.3 カスタム証明書の導入 58
14.2 SSH の強化 59
14.2.1 鍵認証の設定 59
14.2.2 パスワード認証の無効化 59
14.2.3 SSH ポートの変更（任意） 59
14.3 ネットワークセキュリティ 59
14.3.1 管理ネットワークの分離 59
14.3.2 ファイアウォールの推奨ルール 60
14.4 二要素認証（2FA） 60
14.4.1 TOTP の設定 60
14.4.2 WebAuthn / FIDO2 61
14.5 API トークンのセキュリティ 61
14.6 監査ログ 61
14.7 セキュリティチェックリスト 61

1 はじめに

1.1 Proxmox VE とは

Proxmox VE（Virtual Environment）は、オープンソースの仮想化管理プラットフォームです。Debian GNU/Linux をベースに、KVM（カーネルベース仮想マシン）と LXC（Linux コンテナ）の両方を統合管理できる環境を提供します。

主な特徴は以下の通りです：

- **オープンソース**：AGPLv3 ライセンスで無償利用可能（商用サポートは有償）
- **Web UI**：ブラウザベースの管理インターフェースで直感的に操作できる
- **KVM 仮想化**：Windows、Linux、BSD など多様な OS を完全仮想化で動作
- **LXC コンテナ**：軽量なコンテナ型仮想化でリソース効率を最大化
- **クラスタ対応**：複数ノードをクラスタ化し、高可用性（HA）を実現
- **統合ストレージ**：ZFS、Ceph、NFS、iSCSI など多様なストレージバックエンドをサポート
- **バックアップ**：統合バックアップツール（vzdump）と Proxmox Backup Server 連携
- **REST API**：自動化やスクリプトによる管理が可能

1.2 なぜ Proxmox VE を選ぶのか

1.2.1 他の仮想化プラットフォームとの比較

項目	Proxmox VE	VMware ESXi	Hyper-V
ライセンス	オープンソース（無償）	有償（無償版は制限あり）	Windows Server に付属
ハイパーバイザー	KVM + LXC	独自（vmkernel）	独自
管理 UI	Web UI（標準）	vSphere Client	Hyper-V マネージャー
コンテナ	LXC 対応	非対応	非対応
ストレージ	ZFS, Ceph 統合	vSAN（有償）	S2D
コスト	低い	高い	中程度

Proxmox VE は特に以下のようなユースケースに適しています：

- ホームラボでの学習・検証環境の構築
- 中小企業のサーバー仮想化
- 開発・テスト環境の構築
- プライベートクラウドの基盤

1.3 このガイドについて

本書は Proxmox VE を初めて使う方を対象に、インストールから基本操作、 仮想マシン・コンテナの管理、ストレージ・ネットワークの設定、 クラスタ構成までを網羅的に解説する入門書です。

1.4 対象読者

- Linux の基本操作に慣れている方
- 仮想化技術に興味がある方
- 自宅サーバーやホームラボを構築したい方
- VMware からの移行を検討している方

1.5 前提知識

- 基本的な Linux コマンド操作（`cd`、`ls`、`ip` など）
- ネットワークの基礎知識（IP アドレス、サブネット、VLAN など）
- 仮想化の基本概念（仮想マシン、ハイパーバイザーなど）の理解があると望ましい

2 インストール

2.1 システム要件

Proxmox VE を動作させるためのハードウェア要件は以下の通りです。

2.1.1 最小要件

- **CPU**：64bit（Intel EMT64 または AMD64）、Intel VT/AMD-V 対応
- **メモリ**：2 GB 以上（推奨 4 GB 以上、ゲスト OS 分は別途必要）
- **ストレージ**：32 GB 以上（推奨 SSD）
- **ネットワーク**：1 つ以上の NIC

2.1.2 推奨要件

- **CPU**：マルチコア（4 コア以上）、VT-d/AMD-Vi 対応（PCI パススルー用）
- **メモリ**：16 GB 以上（ECC メモリ推奨）
- **ストレージ**：OS 用 SSD + データ用ストレージ（ZFS 利用時は ECC メモリ推奨）
- **ネットワーク**：2 つ以上の NIC（管理用と VM 用で分離）

2.2 ISO イメージのダウンロード

Proxmox VE の ISO イメージは公式サイトからダウンロードします。

1. ブラウザで `https://www.proxmox.com/en/downloads` にアクセス
2. 「Proxmox VE ISO Installer」の最新版をダウンロード
3. ダウンロード後、チェックサムを検証

```
# SHA256 チェックサムの検証
sha256sum proxmox-ve_8.x-x.iso
```

2.3 インストールメディアの作成

2.3.1 USB メモリへの書き込み

```
# Linux の場合
dd bs=1M conv=fdatasync if=proxmox-ve_8.x-x.iso of=/dev/sdX

# macOS の場合
sudo dd if=proxmox-ve_8.x-x.iso of=/dev/rdiskN bs=1m
```

Windows の場合は Rufus や Etcher などのツールを使用してください。

2.4 インストール手順

2.4.1 BIOS/UEFI の設定

インストール前に以下の設定を確認・変更してください：

1. **Intel VT-x / AMD-V**：有効化（仮想化支援）
2. **Intel VT-d / AMD-Vi**：有効化（PCI パススルー用、任意）
3. **ブート順序**：USB を最優先に設定

2.4.2 インストーラーの起動

1. USB メモリからブート
2.「Install Proxmox VE (Graphical)」を選択
3. EULA に同意

2.4.3 ディスクの設定

ターゲットディスクを選択します。「Options」ボタンで詳細設定が可能です：

- **ext4**：一般的なファイルシステム（シンプル）
- **xfs**：高パフォーマンス
- **ZFS（RAID0/1/10/RAIDZ）**：データ保護・スナップショット機能（推奨）

ZFS を選択する場合は複数ディスクの構成を設定できます。

2.4.4 ネットワークの設定

以下を入力します：

- **Management Interface**：管理用 NIC を選択
- **Hostname (FQDN)**：`pve.local` など
- **IP Address**：固定 IP を設定（例：`192.168.1.100/24`）
- **Gateway**：デフォルトゲートウェイ
- **DNS Server**：DNS サーバーのアドレス

2.4.5 パスワードとメールの設定

- **Password**：root ユーザーのパスワード
- **E-mail**：管理者のメールアドレス（通知用）

2.4.6 インストールの完了

設定を確認して「Install」をクリックすると、インストールが開始されます。完了後、再起動するとコンソールに Web UI の URL が表示されます。

2.5 初期設定

2.5.1 Web UI へのアクセス

ブラウザで以下の URL にアクセスします：

```
https://<IPアドレス>:8006
```

自己署名証明書のため、ブラウザの警告が表示されますが続行してください。ログイン情報は以下の通りです：

- **ユーザー名**：`root`
- **レルム**：`Linux PAM standard authentication`
- **パスワード**：インストール時に設定したパスワード

2.5.2 サブスクリプションの無効化（任意）

無償で利用する場合、エンタープライズリポジトリを無効化し、no-subscription リポジトリを有効化します。

```
# エンタープライズリポジトリを無効化
mv /etc/apt/sources.list.d/pve-enterprise.list \
   /etc/apt/sources.list.d/pve-enterprise.list.bak

# no-subscription リポジトリを追加
echo "deb http://download.proxmox.com/debian/pve bookworm pve-no-subscription" \
  > /etc/apt/sources.list.d/pve-no-subscription.list

# パッケージの更新
apt update && apt full-upgrade -y
```

2.5.3 タイムゾーンと NTP の確認

```
# タイムゾーンの確認・設定
timedatectl set-timezone Asia/Tokyo

# NTP の確認（chrony が標準）
systemctl status chronyd
# chronyd が見つからない場合は systemd-timesyncd を確認
# systemctl status systemd-timesyncd
```

3 基本操作

3.1 Web UI の構成

Proxmox VE の Web UI は以下の主要エリアで構成されています：

- **ヘッダーバー**：ユーザー情報、ドキュメントリンク、ログアウト
- **リソースツリー（左側）**：データセンター、ノード、VM、コンテナの階層表示
- **コンテンツパネル（中央）**：選択した項目の詳細情報と操作
- **タスクログ（下部）**：実行中・完了済みタスクの履歴

3.2 ノードの管理

3.2.1 ノード情報の確認

リソースツリーでノードを選択すると、以下の情報を確認できます：

- **Summary**：CPU、メモリ、ストレージの使用状況
- **Notes**：ノードに関するメモ
- **Shell**：Web ベースのコンソール（noVNC / xterm.js）
- **System**：ネットワーク、DNS、時刻、Syslog の設定
- **Updates**：パッケージの更新状態

3.2.2 シェルアクセス

Web UI から直接シェルにアクセスする方法は 2 つあります：

1. **Web UI のシェル**：ノードを選択 → 「Shell」ボタン
2. **SSH 接続**：

```
ssh root@<IPアドレス>
```

3.3 コマンドラインツール

Proxmox VE には専用の CLI ツールが用意されています。

3.3.1 qm（仮想マシン管理）

```
# VM 一覧
qm list

# VM の起動
qm start <vmid>

# VM の停止
```

```
qm stop <vmid>

# VM のシャットダウン（ゲスト OS 経由）
qm shutdown <vmid>

# VM の設定確認
qm config <vmid>

# VM の作成
qm create <vmid> --name <名前> --memory 2048 --cores 2
```

3.3.2 pct（コンテナ管理）

```
# コンテナ一覧
pct list

# コンテナの起動
pct start <ctid>

# コンテナの停止
pct stop <ctid>

# コンテナの設定確認
pct config <ctid>

# コンテナにログイン
pct enter <ctid>
```

3.3.3 pvesm（ストレージ管理）

```
# ストレージ一覧
pvesm status

# ストレージの追加（NFS の例）
pvesm add nfs <ストレージ名> --server <サーバー> --export <パス>
```

3.3.4 pvesh（API シェル）

```
# ノード情報の取得
pvesh get /nodes

# VM 一覧の取得
pvesh get /nodes/<ノード名>/qemu

# リソースの取得
pvesh get /cluster/resources
```

3.4 ISO イメージのアップロード

VM にインストールする OS の ISO イメージをアップロードします。

3.4.1 Web UI から

1. データセンター → ストレージ → `local` を選択
2. 「ISO Images」タブを選択
3. 「Upload」ボタンからファイルを選択
4. アップロードが完了するまで待機

3.4.2 コマンドラインから

```
# wget で直接ダウンロード
cd /var/lib/vz/template/iso/
wget https://example.com/os-image.iso
```

3.5 テンプレートのダウンロード

LXC コンテナ用のテンプレートをダウンロードします。

3.5.1 Web UI から

1. ストレージ → `local` → 「CT Templates」タブ
2. 「Templates」ボタンをクリック
3. 使用したいテンプレートを選択してダウンロード

3.5.2 コマンドラインから

```
# テンプレート一覧の更新
pveam update

# 利用可能なテンプレートの表示
pveam available

# テンプレートのダウンロード
pveam download local debian-12-standard_12.2-1_amd64.tar.zst
```

4 仮想マシン

4.1 仮想マシンの作成

4.1.1 Web UI から作成

1. 右上の「Create VM」ボタンをクリック
2. 各タブで設定を行う：

General
- **Node**：VM を配置するノード
- **VM ID**：一意の番号（デフォルトで自動採番）
- **Name**：VM の名前

OS
- **ISO image**：インストールに使用する ISO を選択
- **Guest OS Type**：Linux / Windows / Other

System
- **Machine**：q35（推奨）または i440fx
- **BIOS**：OVMF（UEFI）または SeaBIOS
- **SCSI Controller**：VirtIO SCSI single（推奨）
- **Qemu Agent**：有効化推奨

Disks
- **Bus/Device**：VirtIO Block または SCSI
- **Storage**：ディスクの保存先
- **Disk size**：ディスクサイズ（GiB）
- **Discard**：SSD 使用時はチェック（TRIM サポート）

CPU
- **Cores**：割り当てるコア数
- **Type**：host（最高性能、ライブマイグレーション時は同一 CPU が必要）または x86-64-v2-AES（異なる CPU 間でのマイグレーション互換性重視）

Memory
- **Memory (MiB)**：割り当てるメモリ量
- **Ballooning Device**：動的メモリ管理（有効化推奨）

Network
- **Bridge**：`vmbr0`（デフォルト）
- **Model**：VirtIO（推奨）
- **VLAN Tag**：必要に応じて設定

4.1.2 コマンドラインから作成

```
# 基本的な VM 作成
qm create 100 \
  --name my-vm \
  --memory 4096 \
  --cores 2 \
  --scsihw virtio-scsi-single \
  --scsi0 local-lvm:32 \
  --ide2 local:iso/debian-12.iso,media=cdrom \
  --net0 virtio,bridge=vmbr0 \
  --boot order=ide2

# VM の起動
qm start 100
```

4.2 VirtIO ドライバー

4.2.1 VirtIO とは

VirtIO は KVM 環境に最適化された準仮想化ドライバーです。ハードウェアをエミュレーションする代わりに、ゲストとホストが効率的に通信するため、ディスク I/O やネットワーク性能が大幅に向上します。

VM 作成時のディスクバスの選択：

- **VirtIO SCSI**：推奨。SCSI コマンドに対応し、ホットプラグや TRIM をサポート
- **VirtIO Block**：シンプルだがホットプラグ非対応。レガシー用途向け
- **IDE / SATA**：VirtIO ドライバーがない場合の互換モード（低性能）

4.2.2 Linux ゲスト

Linux カーネルには VirtIO ドライバーが標準で含まれているため、 追加のインストールは不要です。

4.2.3 Windows ゲスト

Windows には VirtIO ドライバーが含まれていないため、別途インストールが必要です。 インストールしない場合、VirtIO デバイスが認識されません。

1. VirtIO ISO をダウンロード（`https://fedorapeople.org/groups/virt/virtio-win/direct-downloads/stable-virtio/virtio-win.iso`）
2. VM に第二の CD ドライブとして VirtIO ISO をマウント
3. Windows インストール時にドライバーを読み込み、またはインストール後に実行

```
# VirtIO ISO を第二ドライブに追加
qm set 100 --ide2 local:iso/virtio-win.iso,media=cdrom
```

4.3 QEMU Guest Agent

QEMU Guest Agent を導入すると、ホストからゲスト OS を適切に管理できます。

4.3.1 有効化と導入

```
# VM 側で Guest Agent を有効化
qm set <vmid> --agent enabled=1

# Linux ゲストへのインストール
apt install qemu-guest-agent    # Debian/Ubuntu
dnf install qemu-guest-agent    # RHEL/Fedora

systemctl enable --now qemu-guest-agent
```

Guest Agent を有効化すると以下が可能になります：

- IP アドレスの正確な表示
- ファイルシステムの静止（freeze/thaw）によるバックアップの整合性向上
- ゲスト OS の適切なシャットダウン

4.4 リソース配分のガイドライン

4.4.1 CPU

- **コア数**：ゲスト OS の用途に合わせて割り当てる。物理コア数を超える割り当て（オーバーコミット）も可能だが、性能低下に注意
- **CPU ピンニング**：リアルタイム性が求められるワークロードでは `affinity` で特定コアに固定
- **NUMA**：マルチソケットサーバーでは NUMA を有効化すると、メモリアクセスの局所性が向上

```
# CPU アフィニティの設定（コア 0-3 に固定）
qm set <vmid> --affinity 0-3

# NUMA の有効化
qm set <vmid> --numa 1
```

4.4.2 メモリ

- **固定割り当て**：データベースや Java アプリなど、安定したメモリが必要な場合
- **Ballooning**：メモリの動的管理。未使用メモリをホストに返還する。Web サーバーなど負荷が変動するワークロード向き
- **オーバーコミット**：Ballooning を使えば物理メモリ以上の割り当てが可能だが、スワップ発生時に大幅な性能低下

```
# Ballooning の最小メモリを設定
qm set <vmid> --balloon 1024    # 最小 1GB まで縮小可能
```

4.4.3 用途別の目安

用途	CPU	メモリ	ディスク
軽量 Web サーバー	1-2 コア	1-2 GB	10-20 GB
アプリケーションサーバー	2-4 コア	4-8 GB	30-50 GB
データベースサーバー	4-8 コア	8-32 GB	50-200 GB（SSD 推奨）
デスクトップ（Linux）	2-4 コア	4-8 GB	30-50 GB
デスクトップ（Windows）	4+ コア	8+ GB	60+ GB（SSD 推奨）

4.5 スナップショット

スナップショットは VM の状態を保存し、後から復元できる機能です。設定変更やアップデート前の保険として便利ですが、長期間の保持には向きません。

4.5.1 注意点

- スナップショットは差分データを蓄積するため、長期間保持するとストレージを圧迫する
- スナップショットが多いほど I/O 性能が低下する可能性がある
- バックアップの代替にはならない（同一ストレージに依存するため）
- RAM を含むスナップショット（VM 稼働中）はメモリ分のサイズが加算される

```
# スナップショットの作成
qm snapshot <vmid> <スナップショット名> --description "説明"

# スナップショット一覧
qm listsnapshot <vmid>

# スナップショットへのロールバック
qm rollback <vmid> <スナップショット名>

# スナップショットの削除（不要になったら早めに削除）
qm delsnapshot <vmid> <スナップショット名>
```

4.6 クローン

既存の VM を複製する方法は 2 種類あります。

4.6.1 フルクローン

ディスクを完全にコピーします。元の VM とは独立した VM が作成されます。

```
qm clone <元vmid> <新vmid> --name <名前> --full
```

4.6.2 リンクドクローン

ベースイメージを共有し、差分のみ保存します。ディスク使用量を節約できます。

```
# 先にテンプレートに変換
qm template <vmid>

# リンクドクローンの作成
qm clone <元vmid> <新vmid> --name <名前>
```

4.7 テンプレート

頻繁に同じ構成の VM を作成する場合、テンプレート化が便利です。

1. VM を通常通り作成・設定
2. OS のインストールと初期設定を完了
3. `cloud-init` パッケージをインストール（任意）
4. VM をシャットダウン
5. テンプレートに変換

```
qm template <vmid>
```

テンプレートからは「Clone」で新しい VM を作成できます。

4.8 Windows VM のベストプラクティス

Windows をゲスト OS として使用する場合の推奨設定：

1. **Machine**：q35（最新のチップセットエミュレーション）
2. **BIOS**：OVMF（UEFI）— Secure Boot も対応
3. **SCSI Controller**：VirtIO SCSI single
4. **Disk**：VirtIO SCSI（VirtIO ドライバー ISO を CD に追加してからインストール）
5. **Network**：VirtIO
6. **Display**：VirtIO-GPU（3D アクセラレーション不要の場合）
7. **Qemu Agent**：有効化（ゲスト内で `virtio-win-guest-tools.exe` をインストール）

4.8.1 Windows インストール手順

1. VM を作成し、OS の ISO と VirtIO ドライバー ISO の 2 つを CD ドライブにマウント
2. インストーラーでディスクが表示されない場合、「ドライバーの読み込み」→ VirtIO ISO の `vioscsi` フォルダーを選択
3. インストール完了後、VirtIO ISO 内の `virtio-win-guest-tools.exe` を実行して全ドライバーと Guest Agent を一括インストール

5 コンテナ（LXC）

5.1 LXC コンテナとは

LXC（Linux Containers）は、Linux カーネルの機能（namespaces、cgroups）を利用した OS レベルの仮想化技術です。仮想マシンと比較して以下の利点があります：

- **軽量**：カーネルを共有するためオーバーヘッドが少ない
- **高速起動**：数秒で起動
- **低リソース消費**：メモリ・CPU の使用効率が良い
- **ほぼネイティブな性能**：ハードウェアエミュレーションが不要

一方で以下の制約があります：

- Linux カーネルを共有するため、Linux OS のみ動作可能。Windows や BSD は動作不可
- カーネルモジュールのロードなど、一部の操作が制限される
- カーネルバージョンはホストに依存する

5.2 コンテナの作成

5.2.1 Web UI から作成

1. 右上の「Create CT」ボタンをクリック
2. 各タブで設定を行う：

General

- **Node**：コンテナを配置するノード
- **CT ID**：一意の番号
- **Hostname**：ホスト名
- **Password**：root パスワード
- **SSH Public Key**：SSH 公開鍵（任意）

Template

- ダウンロード済みのテンプレートを選択

Root Disk

- **Storage**：ディスクの保存先
- **Disk size**：ルートディスクのサイズ（GiB）

CPU

- **Cores**：割り当てるコア数

Memory

- **Memory (MiB)**：メモリ量
- **Swap (MiB)**：スワップ量

Network

- **Bridge**：`vmbr0`
- **IPv4/IPv6**：DHCP または固定 IP

DNS
- **DNS domain**：ドメイン名
- **DNS servers**：DNS サーバー

5.2.2 コマンドラインから作成

```
# コンテナの作成
pct create 200 local:vztmpl/debian-12-standard_12.2-1_amd64.tar.zst \
  --hostname my-container \
  --memory 1024 \
  --swap 512 \
  --cores 2 \
  --rootfs local-lvm:8 \
  --net0 name=eth0,bridge=vmbr0,ip=dhcp \
  --password

# コンテナの起動
pct start 200
```

5.3 コンテナの管理

5.3.1 基本操作

```
# コンテナ一覧
pct list

# コンテナの起動・停止
pct start <ctid>
pct stop <ctid>
pct shutdown <ctid>

# コンテナに入る
pct enter <ctid>

# コンテナ内でコマンド実行
pct exec <ctid> -- apt update
```

5.3.2 リソースの変更

コンテナはホットプラグに対応しており、稼働中にリソースを変更できます。

```
# メモリの変更
pct set <ctid> --memory 2048
```

```
# CPU コア数の変更
pct set <ctid> --cores 4

# ディスクのリサイズ
pct resize <ctid> rootfs +5G
```

5.4 特権コンテナと非特権コンテナ

5.4.1 非特権コンテナ（推奨）

デフォルトの設定です。UID/GID がマッピングされ、ホストとは異なるユーザー空間で動作します。セキュリティが高く、通常はこちらを使用します。

5.4.2 特権コンテナ

コンテナ内の root がホストの root と同じ権限を持ちます。セキュリティリスクがあるため、特定のアプリケーション（NFS サーバーなど）で 必要な場合にのみ使用してください。

```
# 特権コンテナの作成（--unprivileged 0 で非特権を無効化＝特権モード）
pct create 201 local:vztmpl/debian-12-standard_12.2-1_amd64.tar.zst \
  --unprivileged 0 \
  --hostname privileged-ct \
  --memory 1024 \
  --rootfs local-lvm:8 \
  --net0 name=eth0,bridge=vmbr0,ip=dhcp
```

5.5 マウントポイント

ホストのディレクトリやストレージをコンテナ内にマウントできます。

```
# バインドマウント（ホストのディレクトリを共有）
pct set <ctid> --mp0 /host/path,mp=/container/path

# ストレージからの追加ディスク
pct set <ctid> --mp0 local-lvm:10,mp=/data
```

5.6 コンテナのバックアップと復元

```
# バックアップ
vzdump <ctid> --storage local --compress zstd

# 復元
pct restore <新ctid> /var/lib/vz/dump/vzdump-lxc-<ctid>-*.tar.zst
```

6 ストレージ

6.1 ストレージの概念

Proxmox VE では、ストレージをコンテンツタイプで分類して管理します。

6.1.1 コンテンツタイプ

- **images**：VM のディスクイメージ
- **rootdir**：コンテナのルートディレクトリ
- **iso**：ISO イメージファイル
- **vztmpl**：コンテナテンプレート
- **backup**：バックアップファイル
- **snippets**：スニペット（cloud-init 設定など）

6.2 ストレージの種類

6.2.1 ローカルストレージ

6.2.1.1 LVM

Proxmox VE のデフォルトインストールで使用されるストレージです。

```
# LVM の状態確認
lvs
vgs

# LVM ストレージの追加
pvesm add lvm <名前> --vgname <VG名> --content images,rootdir
```

6.2.1.2 LVM-Thin

シンプロビジョニングに対応した LVM です。スナップショットが可能で、 実際に使用された分だけディスクを消費します。

```
pvesm add lvmthin <名前> --vgname <VG名> --thinpool <プール名> \
  --content images,rootdir
```

6.2.1.3 ZFS

ZFS は高機能なファイルシステム兼ボリュームマネージャーです。

主な特徴：

- **データ整合性**：チェックサムによるデータ破損検出
- **スナップショット**：瞬時に作成、ほぼゼロコスト
- **圧縮**：透過的なデータ圧縮
- **RAID**：ソフトウェア RAID（mirror、raidz、raidz2、raidz3）

- **重複排除**：同一データのデデュプリケーション（1 TB あたり約 5 GB の RAM が必要。通常は無効推奨）

```
# ZFS プールの作成（ミラー：2 台で冗長化）
zpool create -f mypool mirror /dev/sda /dev/sdb

# RAIDZ1（RAID5 相当：3 台以上、1 台障害耐性）
zpool create -f mypool raidz /dev/sda /dev/sdb /dev/sdc

# RAIDZ2（RAID6 相当：4 台以上、2 台障害耐性、推奨）
zpool create -f mypool raidz2 /dev/sda /dev/sdb /dev/sdc /dev/sdd

# ZFS ストレージの追加
pvesm add zfspool <名前> --pool <プール名> --content images,rootdir

# ZFS プールの状態確認
zpool status
zpool list

# ZFS データセットの確認
zfs list
```

6.2.1.4 Directory

通常のディレクトリをストレージとして利用します。

```
# ディレクトリストレージの追加
pvesm add dir <名前> --path /mnt/data --content iso,vztmpl,backup
```

6.2.2 共有ストレージ

6.2.2.1 NFS

ネットワーク経由でファイルを共有するプロトコルです。

```
# NFS ストレージの追加
pvesm add nfs <名前> \
  --server <NFSサーバーIP> \
  --export /share/path \
  --content iso,vztmpl,backup,images
```

6.2.2.2 CIFS/SMB

Windows ファイル共有プロトコルです。

```
pvesm add cifs <名前> \
  --server <サーバーIP> \
  --share <共有名> \
  --username <ユーザー> \
  --content backup,iso,vztmpl
```

6.2.2.3 iSCSI

ブロックデバイスをネットワーク経由で提供するプロトコルです。

```
pvesm add iscsi <名前> \
  --portal <サーバーIP> \
  --target <IQN>
```

6.2.2.4 Ceph (RBD)

分散ストレージシステムです。Proxmox VE に統合されており、 Web UI からクラスタの構築・管理が可能です。

```
pvesm add rbd <名前> \
  --monhost <モニターIP> \
  --pool <プール名> \
  --content images,rootdir
```

6.3 ストレージの管理

```
# ストレージ一覧の確認
pvesm status

# ストレージの詳細
pvesm list <ストレージ名>

# ストレージの削除
pvesm remove <ストレージ名>
```

6.4 推奨構成例

6.4.1 単一ノード構成

- **OS**：ZFS mirror（SSD x 2）
- **VM/CT ディスク**：ZFS（同上またはデータ用プール）
- **ISO・テンプレート**：local ディレクトリ
- **バックアップ**：NFS または外部 HDD

6.4.2 クラスタ構成

- **OS**：各ノードのローカル SSD
- **VM/CT ディスク**：Ceph RBD または共有ストレージ
- **ISO・テンプレート**：NFS（共有）
- **バックアップ**：Proxmox Backup Server

7 ネットワーク

7.1 ネットワークの基本構成

Proxmox VE のネットワークは Linux の標準的なネットワーク機能を使用しています。 デフォルトでは以下のような構成になっています：

- **物理 NIC**（`eno1`、`enp0s3` など）：実際のネットワークインターフェース
- **Linux Bridge**（`vmbr0`）：VM やコンテナが接続する仮想スイッチ

```
物理ネットワーク ─────── eno1 ─────── vmbr0 ──┬── VM 100
                                              ├── VM 101
                                              ├── CT 200
                                              └── ホスト管理
```

7.2 ブリッジの設定

7.2.1 Web UI から

1. ノード → System → Network
2. 「Create」→ 「Linux Bridge」を選択
3. 設定項目を入力して適用

7.2.2 設定ファイル

ネットワーク設定は `/etc/network/interfaces` で管理されます。

```
# /etc/network/interfaces の例
auto lo
iface lo inet loopback

# 物理 NIC
auto eno1
iface eno1 inet manual

# メインブリッジ
auto vmbr0
iface vmbr0 inet static
    address 192.168.1.100/24
    gateway 192.168.1.1
    bridge-ports eno1
    bridge-stp off
    bridge-fd 0
```

設定変更後は以下のコマンドで適用します：

```
ifreload -a
```

7.3 VLAN

VLAN を使用してネットワークを論理的に分離できます。

7.3.1 VLAN Aware ブリッジ

ブリッジを VLAN aware に設定すると、VM ごとに VLAN タグを指定できます。

```
# /etc/network/interfaces
auto vmbr0
iface vmbr0 inet static
    address 192.168.1.100/24
    gateway 192.168.1.1
    bridge-ports eno1
    bridge-stp off
    bridge-fd 0
    bridge-vlan-aware yes
    bridge-vids 2-4094
```

VM のネットワーク設定で VLAN Tag を指定：

```
# VM に VLAN 100 を設定
qm set <vmid> --net0 virtio,bridge=vmbr0,tag=100
```

7.4 ボンディング

複数の NIC を束ねて冗長性と帯域幅を向上させます。

```
# /etc/network/interfaces
auto bond0
iface bond0 inet manual
    bond-slaves eno1 eno2
    bond-miimon 100
    bond-mode 802.3ad       # LACP
    bond-xmit-hash-policy layer3+4

auto vmbr0
iface vmbr0 inet static
    address 192.168.1.100/24
    gateway 192.168.1.1
    bridge-ports bond0
    bridge-stp off
    bridge-fd 0
```

主なボンディングモード：

- **balance-rr (0)**：ラウンドロビン
- **active-backup (1)**：アクティブ/スタンバイ（スイッチ設定不要）
- **802.3ad (4)**：LACP（スイッチ側の設定も必要）

7.5 ファイアウォール

Proxmox VE には統合ファイアウォールが搭載されています。データセンター、ノード、VM/コンテナの 3 階層で設定できます。

7.5.1 ファイアウォールの有効化

```
# データセンターレベルで有効化
# /etc/pve/firewall/cluster.fw
[OPTIONS]
enable: 1

# VM レベルで有効化
# /etc/pve/firewall/<vmid>.fw
[OPTIONS]
enable: 1
```

7.5.2 ルールの追加

Web UI で「Firewall」タブからルールを追加できます。

```
# 設定ファイルの例（/etc/pve/firewall/<vmid>.fw）
[RULES]
IN ACCEPT -source 192.168.1.0/24 -p tcp -dport 22
IN ACCEPT -p tcp -dport 80
IN ACCEPT -p tcp -dport 443
IN DROP
```

7.5.3 セキュリティグループ

よく使うルールセットをグループ化して再利用できます。

```
# /etc/pve/firewall/cluster.fw
[group web-server]
IN ACCEPT -p tcp -dport 80
IN ACCEPT -p tcp -dport 443

[group ssh-access]
IN ACCEPT -source 192.168.1.0/24 -p tcp -dport 22
```

7.6 SDN（Software Defined Networking）

Proxmox VE 8.x では SDN が正式サポートされ、Web UI から ソフトウェア定義のネットワークを構築・管理できます。クラスタ全体で一貫したネットワーク構成を自動的に展開できるのが利点です。

7.6.1 主な概念

- **Zone**：ネットワーク分離の単位。種類によって通信方式が異なる
  - **Simple**：同一ノード内のみ。テスト・開発用途向け
  - **VLAN**：既存の VLAN を使用。物理スイッチとの連携が必要
  - **VXLAN**：L3 ネットワーク上にオーバーレイ。マルチノード向け
  - **EVPN**：BGP ベースの自動ルーティング。大規模環境向け
- **VNet**：Zone 内に作成する仮想ネットワーク。VM/CT のブリッジとして使用
- **Subnet**：VNet 内のサブネット定義（IP 範囲、ゲートウェイ、DHCP）

7.6.2 SDN の基本的な設定手順

1. Web UI の「Datacenter」→「SDN」→「Zones」で Zone を作成
2. 「VNets」で仮想ネットワークを作成し、Zone に関連付け
3. 「Subnets」で IP 範囲を定義
4. 「SDN」画面で「Apply」をクリックして設定を全ノードに展開
5. VM/CT のネットワーク設定で VNet をブリッジとして指定

8 バックアップとリストア

8.1 バックアップの種類

Proxmox VE では `vzdump` ツールを使用してバックアップを取得します。 3 つのバックアップモードがあります：

- **Snapshot**：稼働中の VM/CT をスナップショットベースでバックアップ（推奨）
- **Suspend**：メモリを保存して一時停止後にバックアップ（ダウンタイムあり）
- **Stop**：完全停止後にバックアップ（最も確実だがダウンタイムあり）

8.2 バックアップの実行

8.2.1 Web UI から

1. VM/CT を選択 → 「Backup」タブ
2. 「Backup now」をクリック
3. モード、圧縮方式、ストレージを選択して実行

8.2.2 コマンドラインから

```
# 基本的なバックアップ
vzdump <vmid> --storage local --compress zstd --mode snapshot

# 全 VM/CT のバックアップ
vzdump --all --storage local --compress zstd --mode snapshot

# 複数指定
vzdump 100 101 200 --storage local --compress zstd
```

8.2.3 圧縮方式

- **none**：無圧縮（高速だがサイズ大）
- **lzo**：軽量圧縮（バランス型）
- **gzip**：標準圧縮
- **zstd**：高圧縮率・高速（推奨）

8.3 スケジュールバックアップ

8.3.1 Web UI から設定

1. データセンター → Backup
2. 「Add」をクリック
3. スケジュール、対象、ストレージ、モードを設定

8.3.2 設定ファイル

スケジュールは `/etc/pve/jobs.cfg` で管理されます。

```
job: vzdump
  schedule daily
  all 1
  enabled 1
  storage local
  compress zstd
  mode snapshot
  mailnotification always
  mailto admin@example.com
```

8.4 リストア（復元）

8.4.1 Web UI から

1. ストレージ → 「Backups」タブ
2. バックアップファイルを選択 → 「Restore」

8.4.2 コマンドラインから

```
# VM のリストア
qmrestore /var/lib/vz/dump/vzdump-qemu-100-*.vma.zst 100

# 別の VM ID でリストア
qmrestore /var/lib/vz/dump/vzdump-qemu-100-*.vma.zst 105

# コンテナのリストア
pct restore 200 /var/lib/vz/dump/vzdump-lxc-200-*.tar.zst

# ストレージを指定してリストア
qmrestore /var/lib/vz/dump/vzdump-qemu-100-*.vma.zst 100 \
  --storage local-lvm
```

8.5 Proxmox Backup Server（PBS）

Proxmox Backup Server は Proxmox VE 専用のバックアップサーバーです。 vzdump よりも高度な機能を提供します。

8.5.1 主な特徴

- **増分バックアップ**：変更されたデータのみを転送・保存
- **重複排除**：データストア全体で重複データを排除
- **暗号化**：クライアントサイドの暗号化
- **整合性検証**：自動的なバックアップの整合性チェック

- **高速リストア**：ファイルレベルのリストアも可能

8.5.2 PBS との連携設定

```
# PBS ストレージの追加
pvesm add pbs <名前> \
  --server <PBSサーバーIP> \
  --username <ユーザー>@pbs \
  --datastore <データストア名> \
  --fingerprint <フィンガープリント>
```

8.6 バックアップの保持ポリシー

バックアップの自動削除ルールを設定できます。

```
# 保持設定の例
vzdump <vmid> --storage local \
  --prune-backups keep-daily=7,keep-weekly=4,keep-monthly=6
```

- **keep-last**：直近 N 個を保持
- **keep-daily**：日次で N 日分
- **keep-weekly**：週次で N 週分
- **keep-monthly**：月次で N ヶ月分
- **keep-yearly**：年次で N 年分

9 クラスタ

9.1 クラスタとは

Proxmox VE クラスタは、複数のノードを統合管理する仕組みです。Corosync と pmxcfs を使用して、設定の同期とクラスタ通信を行います。

9.1.1 クラスタの利点

- **一元管理**：すべてのノードを 1 つの Web UI から操作
- **ライブマイグレーション**：稼働中の VM を別ノードに無停止移動
- **高可用性（HA）**：ノード障害時に VM を自動フェイルオーバー
- **共有設定**：ユーザー、権限、ストレージ設定の共有

9.1.2 クラスタの要件

- 全ノードが同じバージョンの Proxmox VE
- 専用のクラスタネットワーク（推奨）
- 全ノード間で名前解決が可能
- 最低 3 ノード（クォーラム確保のため、2 ノードも可能だが非推奨）

9.2 クラスタの構築

9.2.1 最初のノードでクラスタを作成

```
# クラスタの作成
pvecm create <クラスタ名>

# 例
pvecm create my-cluster

# クラスタの状態確認
pvecm status
```

9.2.2 他のノードの参加

```
# 2 台目以降のノードで実行
pvecm add <最初のノードのIP>

# 例
pvecm add 192.168.1.100
```

9.2.3 クラスタの確認

```
# クラスタの状態
pvecm status

# ノード一覧
pvecm nodes

# Corosync の状態確認
systemctl status corosync
```

9.3 ライブマイグレーション

稼働中の VM を別ノードに無停止で移動できます。

9.3.1 前提条件

- 全ノードからアクセス可能な共有ストレージ（またはローカルストレージ間のコピー）
- 互換性のある CPU（同じベンダー推奨）
- 十分なネットワーク帯域

9.3.2 マイグレーションの実行

```
# オンラインマイグレーション（共有ストレージ使用時）
qm migrate <vmid> <移動先ノード> --online

# ローカルストレージ間のマイグレーション
qm migrate <vmid> <移動先ノード> --online --with-local-disks

# コンテナのマイグレーション
pct migrate <ctid> <移動先ノード> --online
```

9.4 高可用性（HA）

HA を設定すると、ノード障害時に VM/CT が自動的に別ノードで再起動されます。

9.4.1 HA の設定

```
# HA リソースの追加
ha-manager add vm:<vmid>

# 優先ノードの設定
ha-manager set vm:<vmid> --group <HAグループ>

# HA の状態確認
ha-manager status
```

9.4.2 HA グループ

HA グループでフェイルオーバー先のノードと優先順位を定義します。

```
# HA グループの作成
ha-manager groupadd prefer-node1 --nodes node1,node2,node3 \
  --nofailback 0 --restricted 0

# restricted: 1 にすると、グループ内のノードでのみ実行
# nofailback: 0 にすると、元のノード復旧時に自動で戻る
```

Web UI では「Datacenter」→「HA」→「Groups」→「Create」から作成できます。

9.4.3 HA の状態とリソース管理

```
# HA リソースの状態確認
ha-manager status

# リソースの状態変更
ha-manager set vm:<vmid> --state started    # 起動状態を維持
ha-manager set vm:<vmid> --state stopped    # 停止状態を維持
ha-manager set vm:<vmid> --state disabled   # HA 管理から一時除外
ha-manager set vm:<vmid> --state ignored    # HA の監視対象外

# HA リソースの削除
ha-manager remove vm:<vmid>
```

9.4.4 フェンシング

HA が正しく機能するためには、フェンシング（障害ノードの強制停止）が重要です。 フェンシングにより、障害ノード上のリソースが安全に別ノードで起動できることを保証します。

Proxmox VE はデフォルトで Linux ソフトウェア watchdog を使用します。 ハードウェア watchdog（IPMI など）を使用するとより信頼性が向上します。

```
# watchdog の状態確認
systemctl status watchdog-mux

# ハードウェア watchdog の設定（/etc/default/pve-ha-manager）
# WATCHDOG_MODULE=ipmi_watchdog
```

9.5 クォーラムの仕組み

クラスタが正常に動作するには過半数のノード（クォーラム）が通信可能である必要があります。クォーラムを失うと、クラスタファイルシステムが読み取り専用になり、VM/CT の起動・停止・マイグレーションが不可能になります。

ノード数	クォーラム	許容障害数
2	2（全台必要）	0（HA 不可）
3	2	1
4	3	1
5	3	2

9.6 2 ノードクラスタ

2 ノード構成ではクォーラムの問題が発生するため、特別な対策が必要です。

9.6.1 QDevice の導入（推奨）

QDevice は外部の軽量サーバーに第三の投票権を持たせ、2 ノードでも安全なクォーラムを実現します。

```
# QDevice サーバー側の準備（Debian/Ubuntu）
apt install corosync-qnetd

# Proxmox ノード側から QDevice をセットアップ
pvecm qdevice setup <QDeviceサーバーIP>

# QDevice の状態確認
pvecm qdevice status
```

QDevice サーバーは低スペックで十分です（Raspberry Pi でも可）。 Proxmox VE 自体をインストールする必要はありません。

9.6.2 QDevice なしの場合

QDevice を使わない場合は、手動でクォーラムを調整します。

```
# 1 ノードがダウンした時に残りのノードで強制的にクォーラムを確保
pvecm expected 1
```

ただし、スプリットブレイン（両ノードが独立稼働）のリスクがあるため、可能な限り QDevice の導入を推奨します。

9.7 クラスタのトラブルシューティング

9.7.1 クラスタの健全性確認

```
# クラスタの状態
pvecm status
```

```
# 全ノードの接続状況
pvecm nodes

# Corosync のリングステータス
corosync-cfgtool -s

# クラスタファイルシステムの状態
pmxcfs -l
```

9.7.2 よくある問題

9.7.2.1 ノードがオフライン表示される

```
# Corosync と pve-cluster の再起動
systemctl restart corosync
systemctl restart pve-cluster

# ネットワーク接続の確認
ping <他ノードのIP>

# Corosync の通信ポート確認（UDP 5405-5412）
ss -ulnp | grep corosync
```

9.7.2.2 クォーラム喪失からの復旧

```
# 残存ノードで期待投票数を調整
pvecm expected 1

# クラスタファイルシステムが復旧するまで待機
pvecm status
```

9.7.2.3 スプリットブレインの解消

1. 両方のノードの VM を確認し、どちらを正とするか決定
2. 一方のノードを停止
3. 稼働ノードでクォーラムを確保
4. 停止ノードを再起動してクラスタに再参加

9.8 クラスタからのノード削除

```
# 1. ノード上の VM/CT を他ノードにマイグレーション
qm migrate <vmid> <移動先ノード> --online

# 2. HA リソースを削除
ha-manager remove vm:<vmid>
```

```
# 3. 残りのノードから削除を実行
pvecm delnode <ノード名>

# 4. 残りのノードで設定をクリーンアップ
rm -rf /etc/pve/nodes/<ノード名>
```

削除するノード自体はクラスタから切り離された後、 Proxmox VE を再インストールするか `pvecm updatecerts` でリセットします。

10 Tips

10.1 Cloud-Init

Cloud-Init を使用すると、VM のデプロイ時にホスト名、ユーザー、SSH 鍵、ネットワーク設定を自動的に適用できます。テンプレートと組み合わせることで、 VM のプロビジョニングを完全に自動化できます。

10.1.1 基本設定

```
# Cloud-Init ドライブの追加
qm set <vmid> --ide2 local-lvm:cloudinit

# ユーザーと認証
qm set <vmid> --ciuser admin
qm set <vmid> --cipassword <パスワード>
qm set <vmid> --sshkeys ~/.ssh/id_rsa.pub

# ネットワーク
qm set <vmid> --ipconfig0 ip=192.168.1.50/24,gw=192.168.1.1
qm set <vmid> --nameserver 8.8.8.8
qm set <vmid> --searchdomain example.com
```

10.1.2 カスタム Cloud-Init 設定

標準のオプションで不足する場合、カスタム YAML を指定できます。

```
# カスタム設定ファイルをスニペットストレージに配置
cat > /var/lib/vz/snippets/my-cloud-init.yml << 'EOF'
#cloud-config
packages:
  - nginx
  - git
  - curl
runcmd:
  - systemctl enable --now nginx
write_files:
  - path: /etc/motd
    content: "Welcome to Proxmox VM\n"
EOF

# カスタム設定を VM に適用
qm set <vmid> --cicustom "vendor=local:snippets/my-cloud-init.yml"
```

10.1.3 Cloud-Init テンプレートのワークフロー

1. クラウドイメージをダウンロード（Ubuntu、Debian など公式提供）

2. VM を作成しディスクをインポート
3. Cloud-Init ドライブとデフォルト設定を追加
4. テンプレートに変換
5. クローン時に Cloud-Init パラメータを変更してデプロイ

```
# Ubuntu クラウドイメージの例
wget https://cloud-images.ubuntu.com/noble/current/noble-server-cloudimg-amd64.img

# VM 作成とディスクインポート
qm create 9000 --name ubuntu-template --memory 2048 --cores 2 \
  --net0 virtio,bridge=vmbr0 --scsihw virtio-scsi-single
qm importdisk 9000 noble-server-cloudimg-amd64.img local-lvm
qm set 9000 --scsi0 local-lvm:vm-9000-disk-0
qm set 9000 --ide2 local-lvm:cloudinit
qm set 9000 --boot order=scsi0
qm set 9000 --serial0 socket --vga serial0
qm set 9000 --agent enabled=1

# デフォルト Cloud-Init 設定
qm set 9000 --ciuser admin --sshkeys ~/.ssh/id_rsa.pub
qm set 9000 --ipconfig0 ip=dhcp

# テンプレート化
qm template 9000

# デプロイ（クローンして個別設定を上書き）
qm clone 9000 110 --name web-server --full
qm set 110 --ipconfig0 ip=192.168.1.110/24,gw=192.168.1.1
qm start 110
```

10.2 PCI パススルー

物理デバイス（GPU など）を VM に直接割り当てることができます。

10.2.1 IOMMU の有効化

```
# /etc/default/grub を編集
# Intel CPU の場合
GRUB_CMDLINE_LINUX_DEFAULT="quiet intel_iommu=on"

# AMD CPU の場合
GRUB_CMDLINE_LINUX_DEFAULT="quiet amd_iommu=on"

# GRUB 更新
update-grub
reboot
```

10.2.2 VFIO モジュールの設定

```
# /etc/modules に追加
cat >> /etc/modules << EOF
vfio
vfio_iommu_type1
vfio_pci
vfio_virqfd
EOF

# カーネルモジュールの更新
update-initramfs -u -k all
reboot
```

10.2.3 デバイスの割り当て

```
# IOMMU グループの確認
find /sys/kernel/iommu_groups/ -type l

# VM にデバイスを追加（Web UI または CLI）
qm set <vmid> --hostpci0 <デバイスID>
```

10.2.4 GPU パススルーの注意点

- IOMMU グループ内の全デバイスをまとめてパススルーする必要がある場合がある
- NVIDIA GPU はデフォルトで仮想環境を検知してドライバーが動作しないことがある。`--hostpci0` に `,x-vga=1` を追加
- ホスト側で使用中の GPU はパススルーできない。ホストはオンボードグラフィックスまたは別 GPU を使用する
- ACS（Access Control Services）対応のマザーボードが望ましい

10.3 REST API の活用

Proxmox VE は完全な REST API を提供しており、自動化に活用できます。

```
# API トークンの作成（Web UI: Datacenter → Permissions → API Tokens）

# curl での API アクセス例
curl -k -H "Authorization: PVEAPIToken=user@pam!token-name=<トークン値>" \
  https://<IP>:8006/api2/json/nodes

# VM 一覧の取得
curl -k -H "Authorization: PVEAPIToken=user@pam!token-name=<トークン値>" \
  https://<IP>:8006/api2/json/nodes/<ノード>/qemu

# VM の起動
curl -k -X POST \
```

```
  -H "Authorization: PVEAPIToken=user@pam!token-name=<トークン値>" \
  https://<IP>:8006/api2/json/nodes/<ノード>/qemu/<vmid>/status/start
```

10.4 Terraform との連携

Terraform の Proxmox プロバイダーを使用して、インフラをコードで管理できます。

```
terraform {
  required_providers {
    proxmox = {
      source  = "bpg/proxmox"
      version = ">= 0.38.0"
    }
  }
}

provider "proxmox" {
  endpoint  = "https://192.168.1.100:8006/"
  api_token = "user@pam!terraform=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
  insecure  = true
}

resource "proxmox_virtual_environment_vm" "example" {
  name      = "terraform-vm"
  node_name = "pve"

  clone {
    vm_id = 9000  # テンプレート VM の ID
  }

  cpu {
    cores = 2
  }

  memory {
    dedicated = 2048
  }

  disk {
    size         = 32
    datastore_id = "local-lvm"
    interface    = "scsi0"
  }

  network_device {
    model  = "virtio"
    bridge = "vmbr0"
  }
}
```

10.5 タグとバルク操作

10.5.1 タグ

VM/CT にタグを付けて分類・検索を効率化できます。

```
# タグの設定
qm set <vmid> --tags "production,web"
pct set <ctid> --tags "development,db"
```

Web UI のリソースツリーでタグによるフィルタリングが可能です。

10.5.2 バルク操作

```
# 特定タグの VM を一括で取得
pvesh get /cluster/resources --type vm | grep "production"

# 全 VM のバックアップ（スケジュールジョブ推奨）
vzdump --all --storage backup-storage --compress zstd --mode snapshot
```

10.6 トラブルシューティング

10.6.1 よくある問題と対処法

10.6.1.1 Web UI にアクセスできない

```
# pveproxy サービスの確認
systemctl status pveproxy

# 再起動
systemctl restart pveproxy

# ファイアウォールの確認
iptables -L -n | grep 8006
```

10.6.1.2 VM が起動しない

```
# VM の設定確認
qm config <vmid>

# ログの確認
journalctl -u pve-qemu-server -f

# タスクログで詳細を確認
pvesh get /nodes/<ノード名>/tasks --limit 5 --errors 1
```

10.6.1.3 ストレージの問題

```
# ZFS プールの状態
zpool status

# LVM の状態
lvs
pvs

# ストレージの再スキャン
pvesm scan <ストレージ種類> <サーバー>
```

10.6.1.4 ロックされた VM/CT

操作中に異常終了した場合、VM がロックされることがあります。

```
# ロックの確認
qm config <vmid> | grep lock

# ロックの解除
qm unlock <vmid>
```

10.7 便利なコマンド集

```
# ノードのリソース使用状況
pvesh get /nodes/<ノード名>/status

# 全 VM/CT のリソース情報
pvesh get /cluster/resources --type vm

# タスクログの確認
pvesh get /nodes/<ノード名>/tasks

# 設定ファイルのバックアップ
tar czf /root/pve-config-$(date +%Y%m%d).tar.gz /etc/pve/

# VM/CT の一覧（全ノード）
pvesh get /cluster/resources --type vm --output-format table

# 特定ノードの VM 一覧
qm list
pct list
```

11 ユーザー管理と認証

11.1 認証レルム

Proxmox VE では、ユーザーの認証方式を「レルム」で管理します。

- **Linux PAM**：ホスト OS の Linux ユーザーで認証。`root@pam` が標準の管理者
- **Proxmox VE Authentication**：Proxmox 独自のユーザーデータベース。OS ユーザー不要
- **LDAP**：LDAP サーバーと連携
- **Active Directory**：Microsoft AD と連携
- **OpenID Connect**：OAuth2/OIDC プロバイダーと連携

11.1.1 レルムの追加（LDAP の例）

```
# LDAP レルムの追加
pveum realm add my-ldap --type ldap \
  --base-dn "dc=example,dc=com" \
  --user-attr "uid" \
  --server1 ldap.example.com \
  --port 389

# Active Directory レルムの追加
pveum realm add my-ad --type ad \
  --domain example.com \
  --server1 ad.example.com \
  --default 0
```

Web UI では「Datacenter」→「Permissions」→「Realms」から設定できます。

11.2 ユーザーの管理

11.2.1 ユーザーの作成

```
# Proxmox VE レルムでユーザーを作成
pveum user add operator@pve --password <パスワード> \
  --firstname "太郎" --lastname "山田" \
  --email taro@example.com

# ユーザー一覧の確認
pveum user list
```

11.2.2 ユーザーの変更・削除

```
# ユーザー情報の変更
pveum user modify operator@pve --email new@example.com

# ユーザーの無効化（削除せず）
pveum user modify operator@pve --enable 0

# ユーザーの削除
pveum user delete operator@pve
```

11.3 グループ

グループを使うと、複数ユーザーにまとめて権限を付与できます。

```
# グループの作成
pveum group add admins --comment "管理者グループ"
pveum group add operators --comment "運用者グループ"

# ユーザーをグループに追加
pveum user modify operator@pve --groups operators

# グループ一覧
pveum group list
```

11.4 ロールと権限

11.4.1 組み込みロール

Proxmox VE にはあらかじめ定義されたロールがあります。

ロール	説明
`Administrator`	全権限
`PVEAdmin`	システム設定以外の管理権限
`PVEVMAdmin`	VM の完全管理
`PVEVMUser`	VM の利用（起動・停止・コンソール）
`PVEDatastoreAdmin`	ストレージの管理
`PVEDatastoreUser`	ストレージの利用（ISO アップロードなど）
`PVEAuditor`	読み取り専用
`NoAccess`	アクセス拒否

11.4.2 権限の付与

権限は「パス（対象）」「ユーザーまたはグループ」「ロール」の組み合わせで設定します。

```
# グループにデータセンター全体の管理権限を付与
pveum acl modify / --groups admins --roles Administrator

# ユーザーに特定 VM のみの操作権限を付与
pveum acl modify /vms/100 --users operator@pve --roles PVEVMUser

# 特定ストレージへのアクセス権を付与
pveum acl modify /storage/local --groups operators --roles PVEDatastoreUser

# 権限一覧の確認
pveum acl list
```

11.4.3 権限パスの構造

権限パスは階層構造になっており、上位の権限は下位に継承されます。

- `/` — データセンター全体
- `/nodes/<ノード名>` — 特定ノード
- `/vms/<vmid>` — 特定 VM/CT
- `/storage/<ストレージ名>` — 特定ストレージ
- `/pool/<プール名>` — リソースプール

11.4.4 カスタムロールの作成

組み込みロールで不足する場合、カスタムロールを作成できます。

```
# VM の起動・停止のみ可能なロール
pveum role add VMOperator --privs "VM.PowerMgmt VM.Console VM.Audit"

# 権限一覧の確認
pveum role list
```

11.5 API トークン

API トークンはスクリプトや外部ツールからの認証に使用します。パスワード認証と異なり、トークンごとに権限を制限できます。

```
# API トークンの作成
pveum user token add root@pam automation --privsep 1

# トークン情報の確認
pveum user token list root@pam
```

- `--privsep 1`：トークンに別途権限を設定（ユーザーの権限を継承しない）

- `--privsep 0`：ユーザーと同じ権限を継承

作成時に表示されるトークン値は一度しか表示されないため、安全に保管してください。

11.6 リソースプール

VM/CT やストレージをグループ化し、まとめて権限管理できます。

```
# プールの作成
pveum pool add development --comment "開発環境"

# VM をプールに追加
qm set 100 --pool development

# コンテナをプールに追加
pct set 200 --pool development

# プールに対して権限を設定
pveum acl modify /pool/development --groups operators --roles PVEVMAdmin
```

11.7 ベストプラクティス

- **root ユーザーを日常的に使わない**：管理者も個人アカウントを作成する
- **グループベースで権限管理**：個別ユーザーへの直接権限付与は避ける
- **最小権限の原則**：必要最低限のロールのみ付与する
- **API トークンは privsep を有効に**：トークンの権限を制限する
- **定期的な棚卸し**：不要なユーザーやトークンは削除する

12 監視と通知

12.1 組み込みの監視機能

12.1.1 Web UI のダッシュボード

ノードを選択すると「Summary」画面で以下をリアルタイム監視できます：

- CPU 使用率（ホスト全体およびゲストごと）
- メモリ使用量
- ネットワークトラフィック
- ディスク I/O
- ストレージ使用率

12.1.2 コマンドラインでの確認

```
# ノードのリソース状況
pvesh get /nodes/<ノード名>/status

# 全 VM/CT のリソース情報
pvesh get /cluster/resources --type vm

# ストレージの使用状況
pvesm status

# ZFS プールの状態
zpool status
zpool list

# ディスクの S.M.A.R.T. 情報
smartctl -a /dev/sda
```

12.2 通知システム

Proxmox VE 8.1 以降では、新しい通知システムが搭載されています。 Web UI の「Datacenter」→「Notifications」から設定します。

12.2.1 通知の仕組み

通知は以下の 3 つの要素で構成されます：

- **Target（送信先）**：メール、Gotify、Webhook など通知の送り先
- **Matcher（条件）**：どのイベントをどの Target に送るかのルール
- **Event（イベント）**：バックアップ完了、フェンシング発生などのトリガー

12.2.2 メール通知の設定

12.2.2.1 Sendmail（デフォルト）

Proxmox VE には Postfix がインストールされています。

```
# Postfix の設定（リレーサーバー経由で送信）
cat >> /etc/postfix/main.cf << EOF
relayhost = [smtp.example.com]:587
smtp_use_tls = yes
smtp_sasl_auth_enable = yes
smtp_sasl_password_maps = hash:/etc/postfix/sasl_passwd
smtp_sasl_security_options = noanonymous
EOF

# SMTP 認証情報の設定
echo "[smtp.example.com]:587 user@example.com:password" \
  > /etc/postfix/sasl_passwd
chmod 600 /etc/postfix/sasl_passwd
postmap /etc/postfix/sasl_passwd

# Postfix の再起動
systemctl restart postfix

# テストメールの送信
echo "Test" | mail -s "PVE Test" admin@example.com
```

12.2.2.2 SMTP Target の作成（Web UI）

1. 「Datacenter」→「Notifications」→「Add」→「SMTP」
2. サーバー情報を入力：
   - **Server**：SMTP サーバーのアドレス
   - **Port**：587（TLS）または 465（SSL）
   - **Username / Password**：認証情報
   - **Mail From**：送信元アドレス
   - **Recipients**：宛先アドレス

12.2.3 Matcher の設定

Matcher でどのイベントをどの Target に送るか定義します。

```
# /etc/pve/notifications.cfg の例
matcher: backup-notify
  target smtp-target
  mode all
  match-severity info,warning,error
  match-field exact:type=vzdump

matcher: ha-notify
  target smtp-target
```

```
  mode all
  match-severity warning,error
  match-field exact:type=fencing
```

主なイベントタイプ：

- `vzdump`：バックアップジョブの結果
- `replication`：レプリケーションの結果
- `fencing`：HA フェンシングの発生
- `package-updates`：パッケージ更新の通知

12.3 外部監視ツールとの連携

12.3.1 Prometheus + Grafana

Proxmox VE のメトリクスを Prometheus で収集し、Grafana で可視化できます。

12.3.1.1 PVE Exporter のセットアップ

```
# 監視サーバー側で pve-exporter をインストール
pip install prometheus-pve-exporter

# 設定ファイルの作成
cat > /etc/pve-exporter.yml << EOF
default:
  user: monitor@pve
  token_name: prometheus
  token_value: "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
  verify_ssl: false
EOF

# 起動
pve_exporter /etc/pve-exporter.yml
```

12.3.1.2 Proxmox 側の準備

```
# 監視用ユーザーの作成
pveum user add monitor@pve
pveum acl modify / --users monitor@pve --roles PVEAuditor

# API トークンの作成
pveum user token add monitor@pve prometheus --privsep 0
```

12.3.1.3 Prometheus の設定

```
# prometheus.yml に追加
scrape_configs:
  - job_name: 'proxmox'
```

```
    static_configs:
      - targets: ['<exporter-host>:9221']
    metrics_path: /pve
    params:
      target: ['<proxmox-host>']
```

12.3.2 SNMP

```
# SNMP の有効化
apt install snmpd
systemctl enable --now snmpd
```

12.4 ヘルスチェックのベストプラクティス

定期的に確認すべき項目：

- **ストレージ**：`zpool status` で degraded がないか
- **ディスク健全性**：`smartctl` で S.M.A.R.T. エラーがないか
- **クラスタ**：`pvecm status` で全ノードがオンラインか
- **バックアップ**：直近のバックアップジョブが成功しているか
- **更新**：セキュリティアップデートが適用されているか
- **証明書**：SSL 証明書の有効期限

```
# 簡易ヘルスチェックスクリプトの例
#!/bin/bash
echo "=== ノード状態 ==="
pvesh get /nodes --output-format text

echo "=== ストレージ状態 ==="
pvesm status

echo "=== ZFS 状態 ==="
zpool status -x

echo "=== 失敗タスク（直近24h）==="
pvesh get /cluster/tasks --errors 1 --limit 10
```

13 アップデートとメンテナンス

13.1 リポジトリの管理

Proxmox VE のパッケージは 3 つのリポジトリから提供されます。

リポジトリ	対象	説明
Enterprise	有償サブスクリプション	安定性最優先。十分にテスト済み
No-Subscription	検証・ホームラボ	テスト済みだが Enterprise より早いリリース
Test	開発者向け	最新だが不安定な可能性あり。本番禁止

13.1.1 リポジトリの確認と設定

```
# 現在のリポジトリ設定を確認
cat /etc/apt/sources.list.d/pve-enterprise.list
cat /etc/apt/sources.list.d/pve-no-subscription.list

# Ceph リポジトリ（Ceph 使用時）
cat /etc/apt/sources.list.d/ceph.list
```

Web UI では「ノード」→「Updates」→「Repositories」で確認・編集できます。

13.2 パッケージの更新

13.2.1 通常のセキュリティ更新

```
# パッケージリストの更新
apt update

# アップグレード可能なパッケージの確認
apt list --upgradable

# パッケージの更新（依存関係の変更を含む）
apt full-upgrade -y

# 不要パッケージの削除
apt autoremove -y
```

Web UI では「ノード」→「Updates」で GUI から実行できます。

13.2.2 更新前の確認事項

1. 重要な VM/CT のバックアップが最新であること

2. クラスタ環境の場合、全ノードが正常であること
3. カーネル更新がある場合は再起動が必要になる

13.2.3 カーネル更新後の再起動

```
# 現在のカーネルバージョン
uname -r

# インストール済みカーネルの確認
dpkg -l | grep pve-kernel

# 再起動（VM は自動停止されるため注意）
reboot
```

クラスタ環境では 1 ノードずつ順番に再起動してください。

13.3 メジャーバージョンアップグレード

Proxmox VE のメジャーアップグレード（例：7.x → 8.x）は慎重に行う必要があります。

13.3.1 アップグレード前の準備

1. **バックアップ**：全 VM/CT および `/etc/pve/` のバックアップ
2. **互換性チェック**：公式のアップグレードチェッカーを実行
3. **ドキュメント確認**：公式のアップグレードガイドを熟読

```
# アップグレードチェッカーの実行（7→8 の例）
pve7to8 --full

# 設定のバックアップ
tar czf /root/pve-config-$(date +%Y%m%d).tar.gz /etc/pve/
```

13.3.2 アップグレード手順の概要

1. 現行バージョンを最新に更新
2. チェッカーの全項目をクリア
3. リポジトリを新バージョンに変更
4. `apt update && apt full-upgrade` を実行
5. 再起動
6. 動作確認

注意：メジャーバージョンの飛ばしはできません（7 → 9 は不可、7 → 8 → 9 の順）。

13.3.3 クラスタ環境でのアップグレード

クラスタ環境では全ノードを同じバージョンにする必要があります。

1. 全ノードの現行バージョンを最新に更新

2. 1 ノードずつ順番にアップグレード
3. 各ノードのアップグレード後に動作確認
4. 全ノード完了後、クラスタの正常性を確認

```
# クラスタの状態確認
pvecm status
pvecm nodes
```

13.4 ノードのメンテナンス

13.4.1 計画メンテナンス

ノードの停止が必要な場合（ハードウェア交換、ファームウェア更新など）の手順：

1. ノード上の VM/CT を他ノードにマイグレーション
2. HA リソースがある場合は HA を一時無効化
3. ノードをシャットダウン
4. メンテナンス実施
5. ノードを起動し、クラスタへの復帰を確認
6. VM/CT を必要に応じて戻す

```
# ノード上の全 VM を一括マイグレーション（手動で 1 台ずつ推奨）
# 対象 VM の確認
qm list

# マイグレーション
qm migrate <vmid> <移動先ノード> --online

# HA の一時無効化
ha-manager set vm:<vmid> --state disabled
```

13.4.2 ディスク交換（ZFS）

ZFS プールのディスクを交換する手順：

```
# 障害ディスクの確認
zpool status

# ディスクの交換（ホットスペアがある場合は自動復旧）
# 新しいディスクを接続後
zpool replace <プール名> <旧デバイス> <新デバイス>

# リシルバー（データ復旧）の進捗確認
zpool status

# 完了後の確認
zpool status
```

13.4.3 ディスク追加（ZFS プール拡張）

```
# ミラーに新しいミラーペアを追加（ストライプ拡張）
zpool add <プール名> mirror /dev/sdc /dev/sdd

# RAIDZ への拡張（RAIDZ vdev の追加、ZFS 2.2+）
zpool add <プール名> raidz /dev/sde /dev/sdf /dev/sdg
```

注意：既存の vdev 内のディスク数は変更できません。拡張は vdev 単位で行います。

13.5 自動化

13.5.1 Unattended Upgrades

セキュリティアップデートを自動適用する設定：

```
apt install unattended-upgrades

# 設定ファイルの編集
cat > /etc/apt/apt.conf.d/50unattended-upgrades << 'EOF'
Unattended-Upgrade::Origins-Pattern {
  "origin=Proxmox,codename=${distro_codename},label=Proxmox Debian Repository";
  "origin=Debian,codename=${distro_codename}-security,label=Debian-Security";
};
Unattended-Upgrade::Mail "admin@example.com";
Unattended-Upgrade::Automatic-Reboot "false";
EOF

# 有効化
dpkg-reconfigure -plow unattended-upgrades
```

カーネルの自動更新は `Automatic-Reboot` を `false` にして、 再起動は手動で計画的に行うことを推奨します。

14 セキュリティ

14.1 SSL/TLS 証明書

14.1.1 デフォルトの自己署名証明書

Proxmox VE はインストール時に自己署名証明書を生成します。 ブラウザの警告を解消するには、正規の証明書を導入します。

14.1.2 Let's Encrypt（ACME）

Proxmox VE には ACME クライアントが統合されています。

```
# ACME アカウントの登録
pvenode acme account register default admin@example.com

# ドメインの設定
pvenode config set --acme domains=pve.example.com

# 証明書の取得
pvenode acme cert order

# 証明書の自動更新はタイマーで管理される
systemctl status pve-daily-update.timer
```

Web UI では「ノード」→「Certificates」→「ACME」から設定できます。

14.1.2.1 DNS チャレンジ（内部ネットワークの場合）

外部からアクセスできない環境では DNS チャレンジを使用します。

```
# DNS プラグインの設定
pvenode acme plugin add dns cloudflare-plugin \
  --type dns --api cf --data "CF_Token=<APIトークン>"

# ドメインの設定（DNS チャレンジ指定）
pvenode config set \
  --acme domains=pve.example.com \
  --acmedomain0 domain=pve.example.com,plugin=cloudflare-plugin
```

14.1.3 カスタム証明書の導入

既存の証明書を手動で設定する場合：

```
# 証明書ファイルの配置
cp fullchain.pem /etc/pve/nodes/<ノード名>/pveproxy-ssl.pem
cp privkey.pem /etc/pve/nodes/<ノード名>/pveproxy-ssl.key
```

```
# pveproxy の再起動
systemctl restart pveproxy
```

14.2 SSH の強化

14.2.1 鍵認証の設定

```
# クライアント側で鍵を生成
ssh-keygen -t ed25519 -C "admin@example.com"

# 公開鍵をサーバーに転送
ssh-copy-id -i ~/.ssh/id_ed25519.pub root@<IPアドレス>

# 鍵認証でログインできることを確認してから、以下を実施
```

14.2.2 パスワード認証の無効化

```
# /etc/ssh/sshd_config を編集
# PasswordAuthentication no
# PermitRootLogin prohibit-password

# 設定の反映
systemctl restart sshd
```

14.2.3 SSH ポートの変更（任意）

```
# /etc/ssh/sshd_config
# Port 22222

systemctl restart sshd
```

ファイアウォールで新しいポートを許可することを忘れないでください。

14.3 ネットワークセキュリティ

14.3.1 管理ネットワークの分離

管理インターフェース（Web UI、SSH）を専用ネットワークに分離することを推奨します。

```
管理ネットワーク（192.168.1.0/24）─── vmbr0 ─── ホスト管理
VM ネットワーク（10.0.0.0/24）    ─── vmbr1 ─── VM/CT トラフィック
ストレージネットワーク（10.0.1.0/24）─── vmbr2 ─── Ceph/NFS 通信
```

```
# /etc/network/interfaces の例
auto vmbr0
iface vmbr0 inet static
    address 192.168.1.100/24
    gateway 192.168.1.1
    bridge-ports eno1
    bridge-stp off
    bridge-fd 0

auto vmbr1
iface vmbr1 inet static
    address 10.0.0.1/24
    bridge-ports eno2
    bridge-stp off
    bridge-fd 0
```

14.3.2 ファイアウォールの推奨ルール

Proxmox ホストへのアクセスを制限する基本ルール：

```
# /etc/pve/firewall/cluster.fw
[OPTIONS]
enable: 1
policy_in: DROP
policy_out: ACCEPT

[RULES]
# Web UI（管理ネットワークのみ）
IN ACCEPT -source 192.168.1.0/24 -p tcp -dport 8006
# SSH（管理ネットワークのみ）
IN ACCEPT -source 192.168.1.0/24 -p tcp -dport 22
# Corosync（クラスタ通信）
IN ACCEPT -source 192.168.1.0/24 -p udp -dport 5405:5412
# ライブマイグレーション
IN ACCEPT -source 192.168.1.0/24 -p tcp -dport 60000:60050
# VNC コンソール
IN ACCEPT -source 192.168.1.0/24 -p tcp -dport 5900:5999
```

14.4 二要素認証（2FA）

Web UI のログインに二要素認証を追加できます。

14.4.1 TOTP の設定

1. Web UI にログイン
2. 右上のユーザー名 → 「TFA」をクリック
3. 「Add」→ 「TOTP」を選択
4. QR コードを認証アプリ（Google Authenticator など）でスキャン

5. 確認コードを入力して有効化

```
# CLI から TOTP を強制する（レルム単位）
pveum realm modify pve --tfa type=totp
```

14.4.2 WebAuthn / FIDO2

ハードウェアセキュリティキー（YubiKey など）にも対応しています。

```
# WebAuthn の設定（Web UI: Datacenter → Permissions → Realms → TFA）
pveum realm modify pve --webauthn-rp "pve.example.com"
```

14.5 API トークンのセキュリティ

```
# トークンには必ず privsep を有効にする
pveum user token add automation@pve ci-token --privsep 1

# トークンに最小限の権限のみ付与
pveum acl modify /vms --users automation@pve --tokens ci-token \
  --roles PVEVMUser

# 不要になったトークンは速やかに削除
pveum user token remove automation@pve ci-token
```

14.6 監査ログ

Proxmox VE の操作ログは syslog に記録されます。

```
# 認証ログの確認
journalctl -u pvedaemon --since "1 hour ago" | grep auth

# タスクログの確認（Web UI の操作履歴）
pvesh get /cluster/tasks --limit 20

# API アクセスログ
journalctl -u pveproxy --since today
```

14.7 セキュリティチェックリスト

- [ ] デフォルトの root パスワードを強固なものに変更
- [ ] 管理者用の個人アカウントを作成し、root の直接利用を避ける
- [ ] SSH 鍵認証を設定し、パスワード認証を無効化
- [ ] SSL 証明書を正規のものに置き換え
- [ ] ファイアウォールで管理ポートへのアクセスを制限

- [ ] 二要素認証を有効化
- [ ] 不要なサービスを停止
- [ ] 定期的にセキュリティアップデートを適用
- [ ] API トークンには最小権限を設定
- [ ] バックアップを暗号化（PBS 使用時）